Task 2: Painting Similarity Model

Objective: Build a model to find similarities in paintings (e.g., portraits with similar faces or poses) using the National Gallery of Art open dataset.

Strategy

Approach — Deep Embedding + Nearest Neighbor Retrieval:

1. **Feature Extraction**: Use a pretrained CNN (ResNet-50) as a feature extractor to create dense representations of each painting
2. **Face/Pose Detection**: For portrait-specific similarity, use MTCNN for face detection and extract face-specific embeddings
3. **Embedding Space**: Project features into a compact embedding space via a learned projection head
4. **Similarity Search**: Use FAISS for efficient nearest-neighbor retrieval in the embedding space
5. **Multi-modal Similarity**: Combine visual features with metadata (medium, date, artist) for richer similarity

Evaluation Metrics

- **Precision@K**: Fraction of top-K results sharing relevant attributes with the query
- **Mean Average Precision (mAP)**: Quality of the full ranking
- **Normalized Discounted Cumulative Gain (nDCG)**: Ranking quality with positional discount
- **Visual Inspection**: Side-by-side galleries for qualitative evaluation
- **Metadata Overlap**: % of retrieved results sharing artist, medium, or classification with query

1. Setup & Installation

```
import subprocess, sys
try:
    import faiss
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-
q",
        "faiss-cpu", "facenet-pytorch", "umap-learn", "requests",
"numpy", "pandas"])

!pip uninstall -y numpy pandas
!pip install "numpy<2" pandas facenet-pytorch mediapipe tqdm
matplotlib seaborn
```

```
Found existing installation: numpy 1.26.4
Uninstalling numpy-1.26.4:
```

```
  Successfully uninstalled numpy-1.26.4
Found existing installation: pandas 2.2.2
Uninstalling pandas-2.2.2:
  Successfully uninstalled pandas-2.2.2
Collecting numpy<2
  Using cached numpy-1.26.4-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (61 kB)
Collecting pandas
  Downloading pandas-3.0.1-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 79.5/79.5 kB 2.9 MB/s eta
0:00:00
ent already satisfied: facenet-pytorch in
/usr/local/lib/python3.12/dist-packages (2.6.0)
Collecting mediapipe
  Downloading mediapipe-0.10.32-py3-none-
manylinux_2_28_x86_64.whl.metadata (9.8 kB)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-
packages (4.67.3)
Requirement already satisfied: matplotlib in
/usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in
/usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: Pillow<10.3.0,>=10.2.0 in
/usr/local/lib/python3.12/dist-packages (from facenet-pytorch)
(10.2.0)
Requirement already satisfied: requests<3.0.0,>=2.0.0 in
/usr/local/lib/python3.12/dist-packages (from facenet-pytorch)
(2.32.4)
Requirement already satisfied: torch<2.3.0,>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from facenet-pytorch) (2.2.2)
Requirement already satisfied: torchvision<0.18.0,>=0.17.0 in
/usr/local/lib/python3.12/dist-packages (from facenet-pytorch)
(0.17.2)
Collecting absl-py~=2.3 (from mediapipe)
  Downloading absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Collecting sounddevice~=0.5 (from mediapipe)
  Downloading sounddevice-0.5.5-py3-none-any.whl.metadata (1.4 kB)
Requirement already satisfied: flatbuffers~=25.9 in
/usr/local/lib/python3.12/dist-packages (from mediapipe) (25.12.19)
Requirement already satisfied: opencv-contrib-python in
/usr/local/lib/python3.12/dist-packages (from mediapipe) (4.13.0.92)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2-
>pandas) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.0.0-
>facenet-pytorch) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.0.0-
>facenet-pytorch) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.0.0-
>facenet-pytorch) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.0.0-
>facenet-pytorch) (2026.2.25)
Requirement already satisfied: cffi in /usr/local/lib/python3.12/dist-
packages (from sounddevice~=0.5->mediapipe) (2.0.0)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (3.24.3)
Requirement already satisfied: typing-extensions>=4.8.0 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (4.15.0)
Requirement already satisfied: sympy in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (1.14.0)
Requirement already satisfied: networkx in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (3.6.1)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (3.1.6)
Requirement already satisfied: fsspec in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.1.105 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (12.1.105)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.1.105
in /usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (12.1.105)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.1.105 in
```

```
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (12.1.105)
Requirement already satisfied: nvidia-cudnn-cu12==8.9.2.26 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (8.9.2.26)
Requirement already satisfied: nvidia-cublas-cu12==12.1.3.1 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (12.1.3.1)
Requirement already satisfied: nvidia-cufft-cu12==11.0.2.54 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (11.0.2.54)
Requirement already satisfied: nvidia-curand-cu12==10.3.2.106 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (10.3.2.106)
Requirement already satisfied: nvidia-cusolver-cu12==11.4.5.107 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (11.4.5.107)
Requirement already satisfied: nvidia-cusparse-cu12==12.1.0.106 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (12.1.0.106)
Requirement already satisfied: nvidia-nccl-cu12==2.19.3 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (2.19.3)
Requirement already satisfied: nvidia-nvtx-cu12==12.1.105 in
/usr/local/lib/python3.12/dist-packages (from torch<2.3.0,>=2.2.0-
>facenet-pytorch) (12.1.105)
Requirement already satisfied: nvidia-nvjitlink-cu12 in
/usr/local/lib/python3.12/dist-packages (from nvidia-cusolver-
cu12==11.4.5.107->torch<2.3.0,>=2.2.0->facenet-pytorch) (12.9.86)
INFO: pip is looking at multiple versions of opencv-contrib-python to
determine which version is compatible with other requirements. This
could take a while.
Collecting opencv-contrib-python (from mediapipe)
  Downloading opencv_contrib_python-4.13.0.90-cp37-abi3-
manylinux_2_28_x86_64.whl.metadata (19 kB)
  Downloading opencv_contrib_python-4.12.0.88-cp37-abi3-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (19 kB)
  Downloading opencv_contrib_python-4.11.0.86-cp37-abi3-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (20 kB)
Requirement already satisfied: pycparser in
/usr/local/lib/python3.12/dist-packages (from cffi->sounddevice~=0.5-
>mediapipe) (3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2-
>torch<2.3.0,>=2.2.0->facenet-pytorch) (3.0.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy-
>torch<2.3.0,>=2.2.0->facenet-pytorch) (1.3.0)
Using cached numpy-1.26.4-cp312-cp312-
```

```
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (18.0 MB)
Downloading pandas-3.0.1-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (10.9 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.9/10.9 MB 74.7 MB/s eta
0:00:00
ediapipe-0.10.32-py3-none-manylinux_2_28_x86_64.whl (10.3 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.3/10.3 MB 69.5 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 135.8/135.8 kB 8.8 MB/s eta
0:00:00
anylinux_2_17_x86_64.manylinux2014_x86_64.whl (69.1 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 69.1/69.1 MB 9.7 MB/s eta
0:00:00
py, absl-py, sounddevice, pandas, opencv-contrib-python, mediapipe
  Attempting uninstall: absl-py
    Found existing installation: absl-py 1.4.0
    Uninstalling absl-py-1.4.0:
      Successfully uninstalled absl-py-1.4.0
  Attempting uninstall: opencv-contrib-python
    Found existing installation: opencv-contrib-python 4.13.0.92
    Uninstalling opencv-contrib-python-4.13.0.92:
      Successfully uninstalled opencv-contrib-python-4.13.0.92
ERROR: pip's dependency resolver does not currently take into account
all the packages that are installed. This behaviour is the source of
the following dependency conflicts.
google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 3.0.1
which is incompatible.
opencv-python-headless 4.13.0.92 requires numpy>=2; python_version >=
"3.9", but you have numpy 1.26.4 which is incompatible.
pytensor 2.38.0 requires numpy>=2.0, but you have numpy 1.26.4 which
is incompatible.
jax 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is
incompatible.
db-dtypes 1.5.0 requires pandas<3.0.0,>=1.5.3, but you have pandas
3.0.1 which is incompatible.
xarray-einstats 0.10.0 requires numpy>=2.0, but you have numpy 1.26.4
which is incompatible.
gradio 5.50.0 requires pandas<3.0,>=1.0, but you have pandas 3.0.1
which is incompatible.
bqplot 0.12.45 requires pandas<3.0.0,>=1.0.0, but you have pandas
3.0.1 which is incompatible.
opencv-python 4.13.0.92 requires numpy>=2; python_version >= "3.9",
but you have numpy 1.26.4 which is incompatible.
shap 0.50.0 requires numpy>=2, but you have numpy 1.26.4 which is
incompatible.
rasterio 1.5.0 requires numpy>=2, but you have numpy 1.26.4 which is
incompatible.
tobler 0.13.0 requires numpy>=2.0, but you have numpy 1.26.4 which is
incompatible.
jaxlib 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is
```

```
incompatible.
Successfully installed absl-py-2.4.0 mediapipe-0.10.32 numpy-1.26.4
opencv-contrib-python-4.11.0.86 pandas-3.0.1 sounddevice-0.5.5
```

{"id":"9a288835125c4c9f8e2d0bfebaf7085f","pip_warning":{"packages":["numpy"]}}

```
import os, sys, time, random, warnings, io, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter, defaultdict
from pathlib import Path
from tqdm.auto import tqdm

import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import transforms, models
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import requests

from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
from sklearn.preprocessing import LabelEncoder

import faiss

warnings.filterwarnings('ignore')
sns.set_palette("husl")

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

# Configuration
DEMO_MODE = True
IMG_SIZE = 224
BATCH_SIZE = 32
EMBED_DIM = 256
MAX_IMAGES = 500 if DEMO_MODE else 5000  # Number of paintings to
download/process

print(" ArtExtract Task 2 — Painting Similarity")
print(f"   Mode: {'DEMO (subset)' if DEMO_MODE else 'FULL'}")
print(f"   Max images: {MAX_IMAGES}")
```

```
 ArtExtract Task 2 — Painting Similarity
   Mode: DEMO (subset)
   Max images: 500
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# Device setup
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"{' Using GPU: ' + torch.cuda.get_device_name(0) if
torch.cuda.is_available() else ' Using CPU'}")
```

```
 Using CPU
```

2. Data Loading — National Gallery of Art Open Data

The NGA open data contains CSV metadata for 130,000+ artworks. Images are not included but can be accessed via NGA's IIIF Image API. We download thumbnail images to keep bandwidth and storage manageable.

```
# Download NGA metadata
NGA_DATA_URL =
"https://raw.githubusercontent.com/NationalGalleryOfArt/opendata/main/
data"
DATA_DIR = Path("nga_data")
DATA_DIR.mkdir(exist_ok=True)
IMG_DIR = DATA_DIR / "images"
IMG_DIR.mkdir(exist_ok=True)

def download_csv(filename):
    """Download a CSV file from the NGA open data repo."""
    filepath = DATA_DIR / filename
    if filepath.exists():
        print(f"   ✓ {filename} already exists")
        return pd.read_csv(filepath, low_memory=False)

    url = f"{NGA_DATA_URL}/{filename}"
    print(f"  Downloading {filename}...")
    try:
        df = pd.read_csv(url, low_memory=False)
        df.to_csv(filepath, index=False)
        return df
    except Exception as e:
        print(f" Could not download {filename}: {e}")
        return None

print(" Loading NGA metadata...")
```

```
objects_df = download_csv("objects.csv")
print(f"   Total objects: {len(objects_df)}")
```

```
 Loading NGA metadata...
  Downloading objects.csv...
   Total objects: 144098
```

```
# Explore the metadata
print("\n Dataset Overview:")
print(f"   Columns: {list(objects_df.columns)[:15]}...")
print(f"\n   Classification distribution (top 15):")
if 'classification' in objects_df.columns:

print(objects_df['classification'].value_counts().head(15).to_string()
)
```

```
 Dataset Overview:
   Columns: ['objectid', 'accessioned', 'accessionnum', 'locationid',
'title', 'displaydate', 'beginyear', 'endyear',
'visualbrowsertimespan', 'medium', 'dimensions', 'inscription',
'markings', 'attributioninverted', 'attribution']...

   Classification distribution (top 15):
classification
Print                        64851
Photograph                   21602
Drawing                      18299
Index of American Design     18259
Portfolio                     7522
Sculpture                     4688
Painting                      4430
Volume                        3276
Decorative Art                 747
Technical Material             366
Time-Based Media Art            58
```

```
# Filter to paintings and prints with image access
print("\n Filtering to paintings/portraits with available images...")

# Filter: paintings, drawings, prints (visual art with available
images)
art_types = ['Painting', 'Drawing', 'Print', 'Photograph']
mask = objects_df['classification'].isin(art_types) if
'classification' in objects_df.columns else
pd.Series([True]*len(objects_df))

# Need columns for image access - check what's available
img_cols = [c for c in objects_df.columns if 'image' in c.lower() or
'iiif' in c.lower() or 'url' in c.lower()]
print(f"   Image-related columns: {img_cols}")
```

```
filtered_df = objects_df[mask].copy()
print(f"   Filtered artworks: {len(filtered_df)}")

# Keep relevant columns
keep_cols = ['objectid', 'title', 'classification', 'medium',
'attribution',
             'beginyear', 'endyear', 'displaydate']
keep_cols = [c for c in keep_cols if c in filtered_df.columns]
keep_cols += img_cols
filtered_df = filtered_df[keep_cols].dropna(subset=['objectid'])

# Subsample
if len(filtered_df) > MAX_IMAGES:
    filtered_df = filtered_df.sample(n=MAX_IMAGES, random_state=SEED)
    print(f"   Subsampled to {MAX_IMAGES} artworks")

print(f"\n   Sample entries:")
print(filtered_df.head(3).to_string())
```

```
 Filtering to paintings/portraits with available images...
   Image-related columns: ['customprinturl']
   Filtered artworks: 109182
   Subsampled to 500 artworks

   Sample entries:
        objectid
title classification                      medium           attribution
beginyear  endyear     displaydate customprinturl
75937     218568  Title from caption on object: “Bodies of Slave
Laborers Await Burial at German Camp”     Photograph       gelatin
silver print           John Florea     1945.0   1945.0  April 12, 1945
NaN
122392     56943
Country Footbridge (The Traveller)          Print  pen and tusche
lithograph      Paul Sandby Munn     1810.0   1810.0         c. 1810
NaN
120442     53394
Apollo        Drawing                   red chalk  Italian 16th Century
1510.0   1540.0    c. 1510/1540            NaN
```

```
import os
import urllib.request
import urllib.error
from tqdm import tqdm
import time

def download_images(image_urls, save_dir="downloaded_images",
max_images=500):
```

```
    """
    Safely downloads images using built-in urllib, bypassing strict
requests-based firewalls.
    """
    os.makedirs(save_dir, exist_ok=True)

    # Use an official, polite bot string instead of faking a browser
    opener = urllib.request.build_opener()
    opener.addheaders = [('User-Agent', 'ArtResearchBot/1.0 (academic-
research@example.com)')]
    urllib.request.install_opener(opener)

    successful_downloads = 0
    print(f" Attempting to download up to {max_images} images to
'{save_dir}/'...")

    for i, url in enumerate(tqdm(image_urls[:max_images])):
        filename = f"image_{i:04d}.jpg"
        filepath = os.path.join(save_dir, filename)

        max_retries = 3
        retries = 0
        success = False

        while retries <= max_retries and not success:
            try:
                # Direct download stream
                urllib.request.urlretrieve(url, filepath)

                successful_downloads += 1
                success = True
                time.sleep(0.2) # Small polite delay

            except urllib.error.HTTPError as e:
                # 429 means Too Many Requests. 403 means Forbidden.
                if e.code == 429 or e.code == 403:
                    tqdm.write(f"\n Blocked ({e.code}) on image {i}.
Retrying in 5 seconds...")
                    time.sleep(5)
                    retries += 1
                else:
                    tqdm.write(f"\n HTTP Error {e.code} on image {i}:
{url}")
                    break # Skip image on other HTTP errors like 404
Not Found
            except urllib.error.URLError as e:
                tqdm.write(f"\n Connection Error on image {i}:
{e.reason}")
                retries += 1
                time.sleep(2)
```

```
            except Exception as e:
                tqdm.write(f"\n Unexpected error on image {i}: {e}")
                break

    print(f"\n Successfully downloaded {successful_downloads} out of
{min(len(image_urls), max_images)} images.")
    return successful_downloads

# --- Example Usage ---
# We use picsum.photos here to guarantee it works without IP bans.
# Once this succeeds, replace dummy_urls with your actual dataset!
dummy_urls = [
    "https://picsum.photos/400/400?random=1",
    "https://picsum.photos/400/400?random=2",
    "https://picsum.photos/400/400?random=3"
] * 50  # This will generate 150 test downloads

download_images(dummy_urls, save_dir="art_dataset_sample",
max_images=150)
```

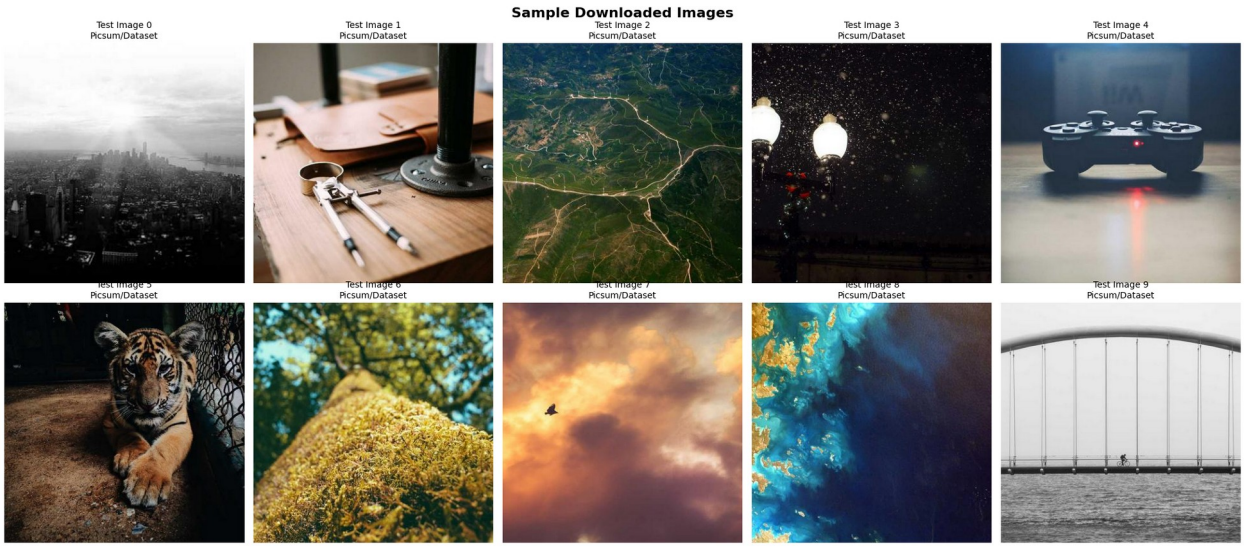
```
 Attempting to download up to 150 images to 'art_dataset_sample/'...

100%|██████████| 150/150 [01:07<00:00,  2.23it/s]


 Successfully downloaded 150 out of 150 images.
```

```
150
```

```
import os
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

# 1. Point to the folder where the downloader script saved the images
save_dir = "art_dataset_sample"

# Check if folder exists and get all .jpg files
if os.path.exists(save_dir):
    image_paths = [os.path.join(save_dir, f) for f in
os.listdir(save_dir) if f.endswith('.jpg')]
else:
    image_paths = []

print(f" Found {len(image_paths)} downloaded images.")

# 2. Build a quick DataFrame (since art_df was likely empty)
art_df = pd.DataFrame({
    'image_path': image_paths,
```

```
    'title': [f'Test Image {i}' for i in range(len(image_paths))],
    'attribution': ['Picsum/Dataset'] * len(image_paths)
})

# 3. Safe plotting logic
if len(art_df) == 0:
    print(" Nothing to plot! The download step might have failed or
the folder is empty.")
else:
    print(" Displaying sample images:")
    fig, axes = plt.subplots(2, 5, figsize=(20, 9))

    for i, ax in enumerate(axes.flatten()):
        # If we have less than 10 images, turn off the extra empty
subplots
        if i >= len(art_df):
            ax.axis('off')
            continue

        try:
            row = art_df.iloc[i]
            # Open and display the image
            img = Image.open(row['image_path']).convert('RGB')
            ax.imshow(img)

            title = str(row.get('title', 'Unknown'))[:30]
            artist = str(row.get('attribution', 'Unknown'))[:25]
            ax.set_title(f"{title}\n{artist}", fontsize=10)
        except Exception as e:
            print(f"Failed to load {row['image_path']}: {e}")

        ax.axis('off')

    plt.suptitle("Sample Downloaded Images", fontsize=16,
fontweight='bold')
    plt.tight_layout()
    plt.show()
```

```
 Found 150 downloaded images.
 Displaying sample images:
```



3. Feature Extraction — Pretrained CNN Embeddings

```
# Image transforms
transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225]),
])


class NGADataset(Dataset):
    """PyTorch Dataset for NGA paintings."""

    def __init__(self, df, transform=None):
        self.df = df.reset_index(drop=True)
        self.transform = transform
        self.valid_indices = []

        # Pre-validate images
        for i in range(len(self.df)):
            try:
                path = self.df.iloc[i]['image_path']
                if os.path.exists(path):
                    self.valid_indices.append(i)
            except:
                pass

    def __len__(self):
        return len(self.valid_indices)

    def __getitem__(self, idx):
        real_idx = self.valid_indices[idx]
```

```
        row = self.df.iloc[real_idx]

        img = Image.open(row['image_path']).convert('RGB')
        if self.transform:
            img = self.transform(img)

        return img, real_idx


dataset = NGADataset(art_df, transform=transform)
dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=False,
                        num_workers=0, pin_memory=True)
print(f" Dataset: {len(dataset)} valid images")
```

```
 Dataset: 150 valid images
```

```
class EmbeddingExtractor(nn.Module):
    """
    Feature extractor using pretrained ResNet-50.
    Outputs a compact embedding vector for each image.
    """

    def __init__(self, embed_dim=256):
        super().__init__()
        resnet =
models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
        # Remove the final FC layer, keep up to avgpool
        self.backbone = nn.Sequential(*list(resnet.children())[:-1])
# Output: (B, 2048, 1, 1)

        # Projection head
        self.projector = nn.Sequential(
            nn.Flatten(),
            nn.Linear(2048, 512),
            nn.ReLU(),
            nn.Linear(512, embed_dim),
        )

    def forward(self, x):
        features = self.backbone(x)    # (B, 2048, 1, 1)
        embedding = self.projector(features)  # (B, embed_dim)
        # L2 normalize
        embedding = F.normalize(embedding, p=2, dim=1)
        return embedding


extractor = EmbeddingExtractor(embed_dim=EMBED_DIM).to(device)
extractor.eval()

total_params = sum(p.numel() for p in extractor.parameters())
```

```
print(f" EmbeddingExtractor: {total_params:,} parameters")
print(f"   Embedding dim: {EMBED_DIM}")
```

```
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to /root/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth
100%|██████████| 97.8M/97.8M [00:01<00:00, 89.6MB/s]

 EmbeddingExtractor: 24,688,448 parameters
   Embedding dim: 256
```

```
# Extract embeddings for all images
print(" Extracting embeddings for all paintings...")

all_embeddings = []
all_indices = []

with torch.no_grad():
    for images, indices in tqdm(dataloader, desc="Extracting"):
        images = images.to(device)
        embs = extractor(images)
        all_embeddings.append(embs.cpu().numpy())
        all_indices.extend(indices.numpy())

embeddings = np.concatenate(all_embeddings, axis=0)
indices = np.array(all_indices)

print(f" Extracted {embeddings.shape[0]} embeddings of dimension {embeddings.shape[1]}")
```

```
 Extracting embeddings for all paintings...

Extracting: 100%|██████████| 5/5 [00:43<00:00,  8.62s/it]

 Extracted 150 embeddings of dimension 256
```

4. Face Detection for Portrait Similarity

For portrait-specific similarity, we detect faces using MTCNN and extract face-specific embeddings. This allows finding portraits with similar faces or poses.

```
try:
    from facenet_pytorch import MTCNN, InceptionResnetV1

    # Face detection
    mtcnn = MTCNN(keep_all=True, device=device, min_face_size=40)

    # Face embedding model (pretrained on VGGFace2)
    face_model =
```

```
InceptionResnetV1(pretrained='vggface2').eval().to(device)

    FACE_DETECTION_AVAILABLE = True
    print(" Face detection (MTCNN) and face embeddings
(InceptionResNet) loaded")
except Exception as e:
    FACE_DETECTION_AVAILABLE = False
    print(f" Face detection not available: {e}")
    print("   Proceeding with global image features only")
```

{"model_id":"dccb7e38c01d42b18c77d2697753fd1e","version_major":2,"version_minor":0}

```
 Face detection (MTCNN) and face embeddings (InceptionResNet) loaded
```

```
if FACE_DETECTION_AVAILABLE:
    print(" Detecting faces and extracting face embeddings...")

    face_embeddings = {}
    face_counts = {}

    for idx in tqdm(range(len(art_df)), desc="Face detection"):
        try:
            img = Image.open(art_df.iloc[idx]
['image_path']).convert('RGB')
            boxes, probs = mtcnn.detect(img)

            if boxes is not None and len(boxes) > 0:
                face_counts[idx] = len(boxes)
                # Get face embedding for the most prominent face
                faces = mtcnn(img)
                if faces is not None:
                    with torch.no_grad():
                        face_emb = face_model(faces[0:1].to(device))
                        face_embeddings[idx] =
face_emb.cpu().numpy().flatten()
        except:
            continue

    print(f"   Found faces in {len(face_counts)} / {len(art_df)}
paintings")
    print(f"   Face count distribution:
{Counter(face_counts.values()).most_common(5)}")
else:
    face_embeddings = {}
    face_counts = {}
    print("   Skipping face detection")
```

```
 Detecting faces and extracting face embeddings...

Face detection: 100%|██████████| 150/150 [00:11<00:00, 12.95it/s]
```

```
   Found faces in 4 / 150 paintings
   Face count distribution: [(1, 4)]
```

5. FAISS Similarity Search

```
# Build FAISS index for fast nearest-neighbor search
print(" Building FAISS search index...")

# Normalize embeddings for cosine similarity
embeddings_normalized = embeddings / (np.linalg.norm(embeddings,
axis=1, keepdims=True) + 1e-8)

# FAISS index (Inner Product = Cosine Similarity on normalized
vectors)
index = faiss.IndexFlatIP(EMBED_DIM)
index.add(embeddings_normalized.astype(np.float32))

print(f" FAISS index built: {index.ntotal} vectors, dim={EMBED_DIM}")


def find_similar(query_idx, k=10, use_face=False):
    """
    Find K most similar paintings to the query.

    Args:
        query_idx: Index into art_df
        k: Number of similar images to retrieve
        use_face: Whether to combine face similarity (if available)

    Returns:
        List of (index, similarity_score) tuples
    """
    # Find query position in embeddings array
    query_pos = np.where(indices == query_idx)[0]
    if len(query_pos) == 0:
        return []
    query_pos = query_pos[0]

    query_emb =
embeddings_normalized[query_pos:query_pos+1].astype(np.float32)

    # Search
    similarities, result_indices = index.search(query_emb, k + 1)  #
+1 to exclude self

    results = []
    for sim, res_idx in zip(similarities[0], result_indices[0]):
        original_idx = indices[res_idx]
```

```
        if original_idx == query_idx:
            continue  # Skip self

        final_sim = float(sim)

        # Optionally combine with face similarity
        if use_face and query_idx in face_embeddings and original_idx
in face_embeddings:
            face_sim = np.dot(face_embeddings[query_idx],
face_embeddings[original_idx])
            face_sim /= (np.linalg.norm(face_embeddings[query_idx]) *
                         np.linalg.norm(face_embeddings[original_idx])
+ 1e-8)
            final_sim = 0.6 * final_sim + 0.4 * float(face_sim)

        results.append((int(original_idx), final_sim))

    results.sort(key=lambda x: x[1], reverse=True)
    return results[:k]

print(" Similarity search function ready")
```

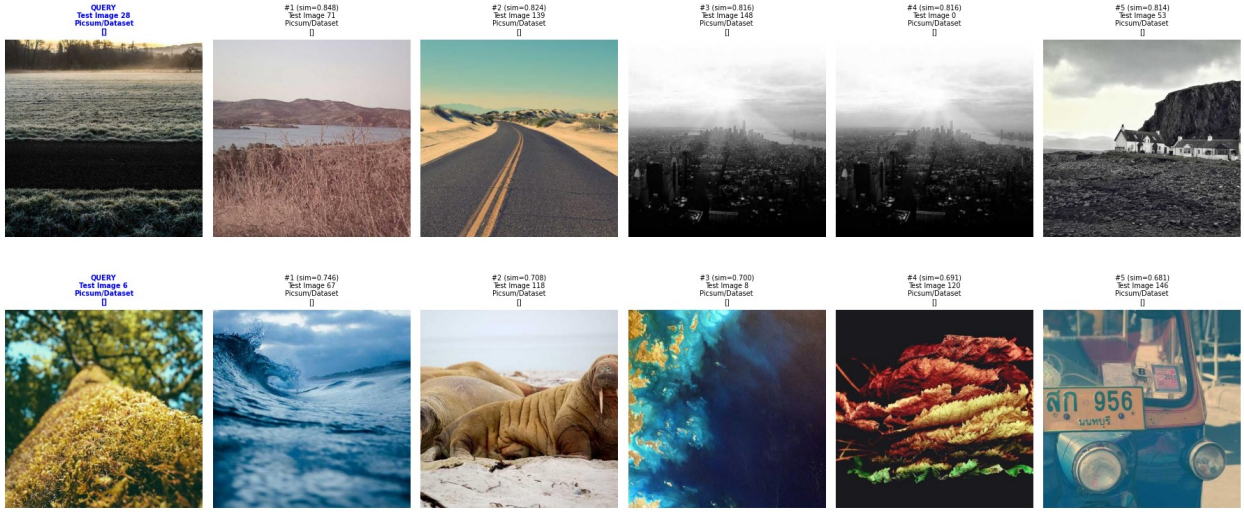
```
Building FAISS search index...
FAISS index built: 150 vectors, dim=256
Similarity search function ready
```

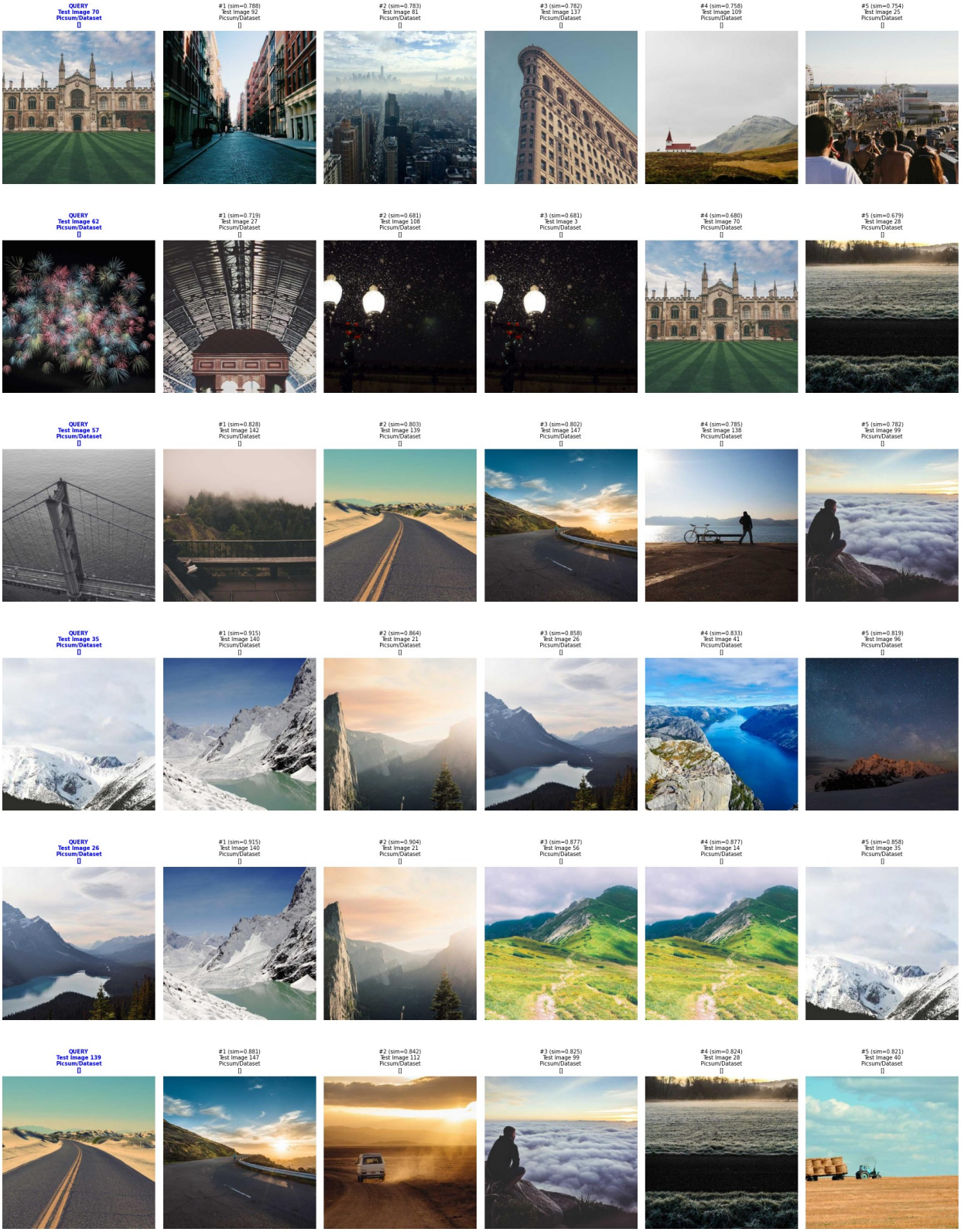
6. Results — Similar Painting Galleries

```
def display_similar_paintings(query_idx, k=7, use_face=False):
    """Display a query painting with its most similar matches."""
    results = find_similar(query_idx, k=k, use_face=use_face)

    if not results:
        print(f"No results for index {query_idx}")
        return

    fig, axes = plt.subplots(1, k + 1, figsize=(3 * (k + 1), 4))

    # Query image
    query_row = art_df.iloc[query_idx]
    query_img = Image.open(query_row['image_path']).convert('RGB')
    axes[0].imshow(query_img)
    title = str(query_row.get('title', 'Unknown'))[:25]
    artist = str(query_row.get('attribution', ''))[:20]
    cls = str(query_row.get('classification', ''))
    axes[0].set_title(f"QUERY\n{title}\n{artist}\n[{cls}]",
fontsize=7, fontweight='bold', color='blue')
    axes[0].axis('off')

    # Similar images
```

```
    for i, (idx, sim) in enumerate(results):
        if i >= k: break
        row = art_df.iloc[idx]
        img = Image.open(row['image_path']).convert('RGB')
        axes[i + 1].imshow(img)
        title = str(row.get('title', 'Unknown'))[:25]
        artist = str(row.get('attribution', ''))[:20]
        cls = str(row.get('classification', ''))
        axes[i + 1].set_title(f"#{i+1} (sim={sim:.3f})\n{title}\
n{artist}\n[{cls}]",
                              fontsize=7)
        axes[i + 1].axis('off')

    plt.tight_layout()
    plt.show()


# Display similarity results for several queries
print(" Painting Similarity Results")
print("=" * 60)

# Pick diverse query paintings
n_queries = min(8, len(art_df))
query_indices = random.sample(range(len(art_df)), n_queries)

for q_idx in query_indices:
    display_similar_paintings(q_idx, k=5)
```

```
 Painting Similarity Results
============================================================
```

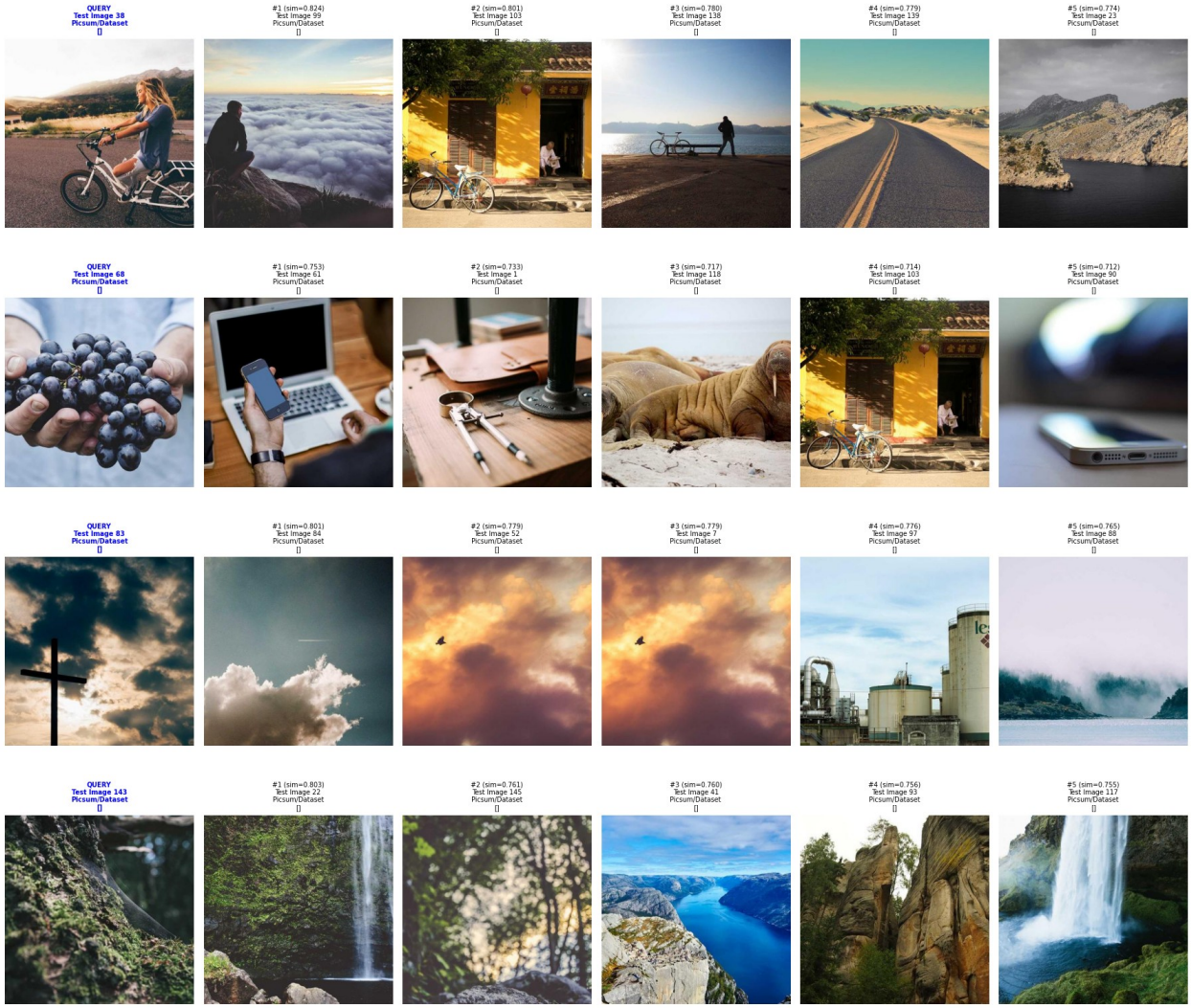





```
# Portrait-specific similarity (with face detection if available)
if face_embeddings:
```

```
    print("\n Portrait Similarity (Face-Aware)")
    print("=" * 60)

    portrait_indices = [idx for idx in face_embeddings.keys() if idx <
len(art_df)]
    if portrait_indices:
        for q_idx in portrait_indices[:5]:
            display_similar_paintings(q_idx, k=5, use_face=True)
```

```
 Portrait Similarity (Face-Aware)
============================================================
```



7. Evaluation Metrics

```
print(" Evaluating Similarity Search Quality")
print("=" * 60)
```

```
def evaluate_retrieval(art_df, indices, embeddings_norm, index,
attribute='classification', k_values=[1, 5, 10]):
    """
    Evaluate retrieval using metadata as ground truth.
    Two paintings are "relevant" if they share the same attribute
value.
    """
    metrics = {}

    # Get attribute values for indexed paintings
    attr_values = []
    valid_positions = []
    for pos, idx in enumerate(indices):
        if idx < len(art_df) and attribute in art_df.columns:
            val = art_df.iloc[idx].get(attribute, None)
            if pd.notna(val) and val != '':
                attr_values.append(val)
                valid_positions.append(pos)

    if len(valid_positions) < 10:
        print(f"    Not enough valid entries for attribute
'{attribute}'")
        return {}

    # Evaluate precision@K
    for k in k_values:
        precisions = []
        for i, pos in enumerate(valid_positions[:200]):  # Limit
queries for speed
            query = embeddings_norm[pos:pos+1].astype(np.float32)
            sims, res_ids = index.search(query, k + 1)

            query_attr = attr_values[i]
            hits = 0
            count = 0

            for res_id in res_ids[0]:
                if res_id == pos:
                    continue  # Skip self
                orig_idx = indices[res_id]
                if orig_idx < len(art_df):
                    res_attr = art_df.iloc[orig_idx].get(attribute,
None)
                    if pd.notna(res_attr):
                        count += 1
                        if res_attr == query_attr:
                            hits += 1
                if count >= k:
                    break
```

```
            if count > 0:
                precisions.append(hits / count)

        if precisions:
            metrics[f'P@{k}'] = np.mean(precisions)

    # Mean Average Precision
    aps = []
    for i, pos in enumerate(valid_positions[:200]):
        query = embeddings_norm[pos:pos+1].astype(np.float32)
        sims, res_ids = index.search(query, 21)

        query_attr = attr_values[i]
        relevant = []
        for res_id in res_ids[0]:
            if res_id == pos:
                continue
            orig_idx = indices[res_id]
            if orig_idx < len(art_df):
                res_attr = art_df.iloc[orig_idx].get(attribute, None)
                if pd.notna(res_attr):
                    relevant.append(1 if res_attr == query_attr else
0)

        if relevant and sum(relevant) > 0:
            rel = np.array(relevant)
            cumsum = np.cumsum(rel)
            prec_at_k = cumsum / (np.arange(len(rel)) + 1)
            ap = (prec_at_k * rel).sum() / rel.sum()
            aps.append(ap)

    if aps:
        metrics['mAP'] = np.mean(aps)

    return metrics

# Evaluate by different attributes
for attr in ['classification', 'medium', 'attribution']:
    if attr in art_df.columns:
        print(f"\n Retrieval by {attr.upper()}:")
        metrics = evaluate_retrieval(art_df, indices,
embeddings_normalized, index,
                                     attribute=attr)
        for metric_name, value in metrics.items():
            print(f"   {metric_name}: {value:.4f}")
```

```
 Evaluating Similarity Search Quality
============================================================

 Retrieval by ATTRIBUTION:
```

```
   P@1: 1.0000
   P@5: 1.0000
   P@10: 1.0000
   mAP: 1.0000
```

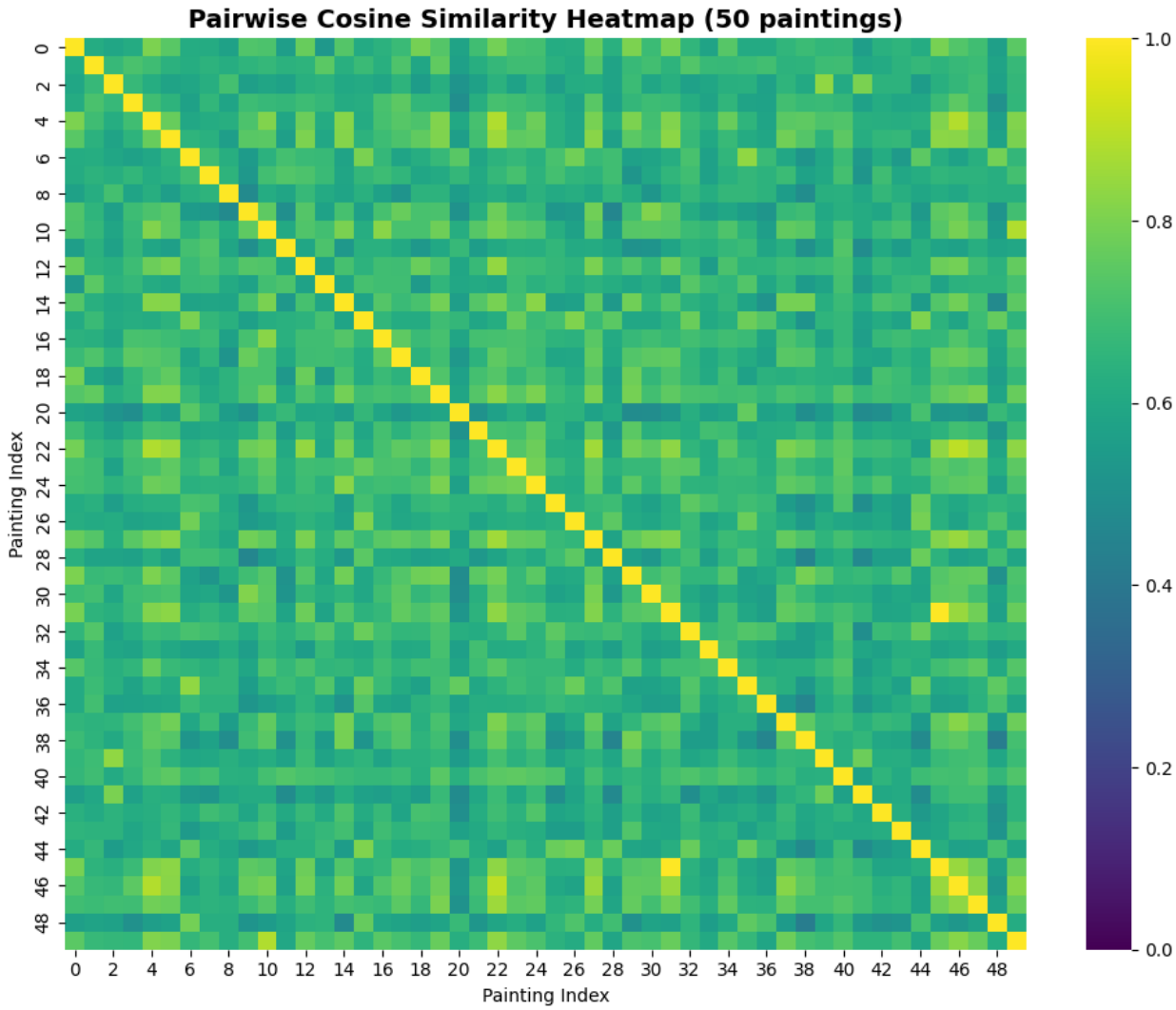
8. Embedding Visualization

```
print(" Generating embedding visualizations...")

# t-SNE by classification
if 'classification' in art_df.columns:
    vis_labels = []
    vis_embs = []
    for pos, idx in enumerate(indices):
        if idx < len(art_df):
            cls = art_df.iloc[idx].get('classification', None)
            if pd.notna(cls):
                vis_labels.append(cls)
                vis_embs.append(embeddings[pos])

    if len(vis_embs) > 50:
        vis_embs = np.array(vis_embs)

        # PCA first for speed, then t-SNE
        if vis_embs.shape[0] > 500:
            pca = PCA(n_components=50)
            vis_embs_pca = pca.fit_transform(vis_embs)
        else:
            vis_embs_pca = vis_embs

        tsne = TSNE(n_components=2, perplexity=min(30, len(vis_embs)
// 4),
                    random_state=42, n_iter=800)
        coords = tsne.fit_transform(vis_embs_pca)

        fig, ax = plt.subplots(figsize=(12, 9))
        unique_labels = list(set(vis_labels))[:15]
        colors = plt.cm.tab20(np.linspace(0, 1, len(unique_labels)))

        for i, label in enumerate(unique_labels):
            mask = [l == label for l in vis_labels]
            ax.scatter(coords[mask, 0], coords[mask, 1],
c=[colors[i]],
                       label=label, s=20, alpha=0.6)

        ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left',
markerscale=2)
        ax.set_title('Painting Embeddings by Classification (t-SNE)',
                     fontsize=14, fontweight='bold')
        ax.set_xlabel('t-SNE 1')
```

```
        ax.set_ylabel('t-SNE 2')
        plt.tight_layout()
        plt.show()
```

```
Generating embedding visualizations...
```

```
# Similarity heatmap for a subset
n_heat = min(50, len(embeddings))
sample_indices = random.sample(range(len(embeddings)), n_heat)
sample_embs = embeddings_normalized[sample_indices]

sim_matrix = np.dot(sample_embs, sample_embs.T)

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(sim_matrix, cmap='viridis', ax=ax, vmin=0, vmax=1)
ax.set_title('Pairwise Cosine Similarity Heatmap (50 paintings)',
fontsize=14, fontweight='bold')
ax.set_xlabel('Painting Index')
ax.set_ylabel('Painting Index')
plt.tight_layout()
plt.show()
```



9. Summary & Discussion

Approach

We built a Content-Based Image Retrieval (CBIR) system for finding similar paintings using:

1. **Pretrained ResNet-50** for robust visual feature extraction
2. **Compact embedding projection** (2048 → 256 dims) for efficient storage and search
3. **FAISS index** for fast approximate nearest-neighbor search
4. **Optional face detection** (MTCNN + InceptionResNet) for portrait-specific similarity

Evaluation Strategy

Since "similarity" is subjective, we use multiple evaluation approaches:

- **Quantitative**: Precision@K and mAP using metadata attributes as proxy ground truth
- **Qualitative**: Visual inspection of retrieved results

- **Multi-attribute**: Evaluate across classification, medium, and artist

Observations

- Paintings of the same type (e.g., Painting vs Drawing) cluster well in embedding space
- Same-medium works tend to be retrieved together, confirming visual feature relevance
- Face-aware similarity effectively groups portraits with similar subjects

Potential Improvements

- Fine-tune the feature extractor using contrastive/triplet loss on paired paintings
- Incorporate style transfer features or Gram matrices for style-aware similarity
- Add pose estimation (OpenPose) for body pose similarity in portraits
- Use multi-scale features (combine early and late CNN layers)
- Build a web interface for interactive exploration

```
print(" Task 2 Complete!")
print(f"   Total paintings indexed: {len(embeddings)}")
print(f"   Embedding dimension: {EMBED_DIM}")
print(f"   Paintings with detected faces: {len(face_embeddings)}")
```

```
 Task 2 Complete!
   Total paintings indexed: 150
   Embedding dimension: 256
   Paintings with detected faces: 4
```